

Predicting 30-Day ICU Readmission from MIMIC-III

Hospitals want to know, as early as possible, which patients are likely to come back after they're discharged. This notebook builds a small model that does exactly that: given some basic facts about a hospital stay, it predicts whether that patient is likely to be readmitted within 30 days.

If any of these terms are new to you, don't worry - they're explained the first time they come up:

- **Readmission:** coming back to the hospital after being discharged.
- **Feature:** a piece of information about a patient or visit that we give to the model (for example, their age, or how many times they've been admitted before).
- **Label:** the thing we're trying to predict (here, whether they come back within 30 days - yes or no).
- **Deep learning:** a type of machine learning that uses a "neural network," a model loosely inspired by how neurons connect in a brain. We'll build a small one and compare it against a simpler, more traditional model.

Data source: MIMIC-III, accessed through Google BigQuery (`physionet-data.mimiciii_clinical`).

1. Install the libraries we need

```
In [1]: # TensorFlow/Keras builds the neural network, scikit-learn builds the simpler
# baseline model, pandas handles the data, and matplotlib/seaborn make the plots.
!pip -q install --upgrade scikit-learn tensorflow matplotlib seaborn pandas
```

```
_____ 80.3/80.3 kB 1.2 MB/s eta 0:00:00
_____ 79.5/79.5 kB 3.4 MB/s eta 0:00:00
_____ 9.1/9.1 MB 41.2 MB/s eta 0:00:00
_____ 572.6/572.6 MB 1.4 MB/s eta 0:00:00
_____ 10.0/10.0 MB 42.4 MB/s eta 0:00:00
_____ 10.9/10.9 MB 44.6 MB/s eta 0:00:00
_____ 4.9/4.9 MB 39.2 MB/s eta 0:00:00
_____ 327.1/327.1 kB 17.5 MB/s eta 0:00:00
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed
. This behaviour is the source of the following dependency conflicts.
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 3.0.3 which is incompatible.
tensorflow-text 2.20.1 requires tensorflow<2.21,>=2.20.0, but you have tensorflow 2.21.0 which is incompati
ble.
ydf-tf 2.20.0 requires tensorflow==2.20.0, but you have tensorflow 2.21.0 which is incompatible.
grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf 7.35.1 which is incompatibl
e.
google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.
5,<6.0.0dev,>=3.20.2, but you have protobuf 7.35.1 which is incompatible.
tf-keras 2.20.0 requires tensorflow<2.21,>=2.20, but you have tensorflow 2.21.0 which is incompatible.
ydf 0.15.0 requires protobuf<7.0.0,>=5.29.1, but you have protobuf 7.35.1 which is incompatible.
```

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    roc_auc_score, roc_curve, confusion_matrix,
    classification_report, ConfusionMatrixDisplay
)

import tensorflow as tf
from tensorflow.keras import layers, models

# setting a fixed seed means the random parts of this notebook (like the
# train/test split, or how the neural network's weights start out) come out
# the same way every time you run it - useful for reproducing results
SEED = 42
```

```
np.random.seed(SEED)
tf.random.set_seed(SEED)

sns.set_style("whitegrid")
```

2. Connect to BigQuery

MIMIC-III is hosted on Google BigQuery. This step logs in and opens a connection so we can run SQL queries against it directly from the notebook.

```
In [3]: from google.colab import auth
from google.cloud import bigquery

auth.authenticate_user()

PROJECT_ID = 'mc-ut-msai-aii-1' # change this to your own BigQuery project ID
client = bigquery.Client(project=PROJECT_ID)

# a small test query just to confirm the connection works
test_query = "SELECT COUNT(*) AS total FROM `physionet-data.mimiciii_clinical.admissions`"
print(client.query(test_query).to_dataframe())

total
0 58976
```

3. Build the dataset with SQL

Here we turn raw MIMIC-III tables into a single, clean table: one row per hospital admission, a set of features describing that admission, and a label saying whether the patient came back within 30 days.

The label: for each admission, we look at the patient's *next* admission (if there is one). If it starts within 30 days of this admission's discharge, we mark this row as a readmission (1). Otherwise it's 0.

The features we use, and why each one might matter:

- `admission_type` - whether the visit was an emergency, urgent, or planned (elective) admission. Emergency patients tend to be sicker or less medically stable going in, so they're a reasonable bet to also come back more often.
- `prior_admission_count` - how many times this patient has been admitted before. Patients with a long admission history are often dealing with a chronic condition, which makes another visit more likely.
- `primary_diagnosis` - the main condition the patient was admitted for. Certain conditions (like sepsis or respiratory failure) are known to relapse more easily than others.
- `lab_abnormal_pct` - what fraction of this patient's lab tests came back outside the normal range. A patient discharged with many abnormal results may have left before their condition was fully under control.
- `age` and `gender` - basic demographics, included as a sanity check more than a strong expectation - we let the model tell us whether they actually matter, rather than assuming it up front.

One MIMIC-specific detail to watch for: to protect patient privacy, MIMIC-III shifts the recorded age of anyone over 89, which can make some patients appear to be over 200 years old in the raw data. We cap age at 90 below so the model doesn't get confused by these values.

```
In [4]: feature_query = """
WITH admissions_ordered AS (
  SELECT
    subject_id, hadm_id, admittance, dischtime, admission_type,
    LEAD(admittime) OVER (PARTITION BY subject_id ORDER BY admittance) AS next_admittime
  FROM `physionet-data.mimiciii_clinical.admissions`
),

labeled AS (
  SELECT
    *,
    CASE
```

```

        WHEN next_admittime IS NOT NULL
        AND DATE_DIFF(DATE(next_admittime), DATE(disctime), DAY) BETWEEN 0 AND 30
        THEN 1 ELSE 0
    END AS readmit_30d
FROM admissions_ordered
),

-- how many admissions did this patient already have before this one?
prior_counts AS (
    SELECT
        subject_id, hadm_id,
        ROW_NUMBER() OVER (PARTITION BY subject_id ORDER BY admittime) - 1 AS prior_admission_count
    FROM labeled
),

-- the main (first-listed) diagnosis for each admission
primary_dx AS (
    SELECT dx.hadm_id, d.short_title
    FROM `physionet-data.mimiciii_clinical.diagnoses_icd` dx
    JOIN `physionet-data.mimiciii_clinical.d_icd_diagnoses` d
        ON dx.icd9_code = d.icd9_code
    WHERE dx.seq_num = 1
),

-- what fraction of this admission's lab tests came back abnormal
lab_summary AS (
    SELECT
        hadm_id,
        COUNT(*) AS total_labs,
        SUM(CASE WHEN flag = 'abnormal' THEN 1 ELSE 0 END) AS abnormal_labs
    FROM `physionet-data.mimiciii_clinical.labevents`
    WHERE hadm_id IS NOT NULL
    GROUP BY hadm_id
)

SELECT
    l.subject_id,
    l.hadm_id,
    l.admission_type,
    l.readmit_30d,
    p.gender,
    -- cap age at 90 because of the de-identification shift described above
    LEAST(DATE_DIFF(DATE(l.admittime), DATE(p.dob), YEAR), 90) AS age,
    pc.prior_admission_count,
    COALESCE(dx.short_title, 'Unknown') AS primary_diagnosis,
    ls.total_labs,
    SAFE_DIVIDE(ls.abnormal_labs, ls.total_labs) AS lab_abnormal_pct
FROM labeled l
JOIN `physionet-data.mimiciii_clinical.patients` p
    ON l.subject_id = p.subject_id
JOIN prior_counts pc
    ON l.hadm_id = pc.hadm_id
LEFT JOIN primary_dx dx
    ON l.hadm_id = dx.hadm_id
LEFT JOIN lab_summary ls
    ON l.hadm_id = ls.hadm_id
WHERE l.admission_type != 'NEWBORN' -- newborn stays behave very differently, so we leave them out
"""

df = client.query(feature_query).to_dataframe()
print(f"Rows: {len(df):,}")
df.head()

```

Rows: 51,113

Out[4]:

	subject_id	hadm_id	admission_type	readmit_30d	gender	age	prior_admission_count	primary_diagnos
0	189	119333	ELECTIVE	0	F	34	0	Cong aorta va insuff
1	1620	128318	EMERGENCY	1	F	30	0	Chr stomach ulc he
2	1620	122881	EMERGENCY	0	F	30	1	Myl sarcoma w achv rms
3	1620	197959	EMERGENCY	0	F	30	2	Hypoxem
4	5032	199413	EMERGENCY	0	F	36	1	Neurosyphilis NC

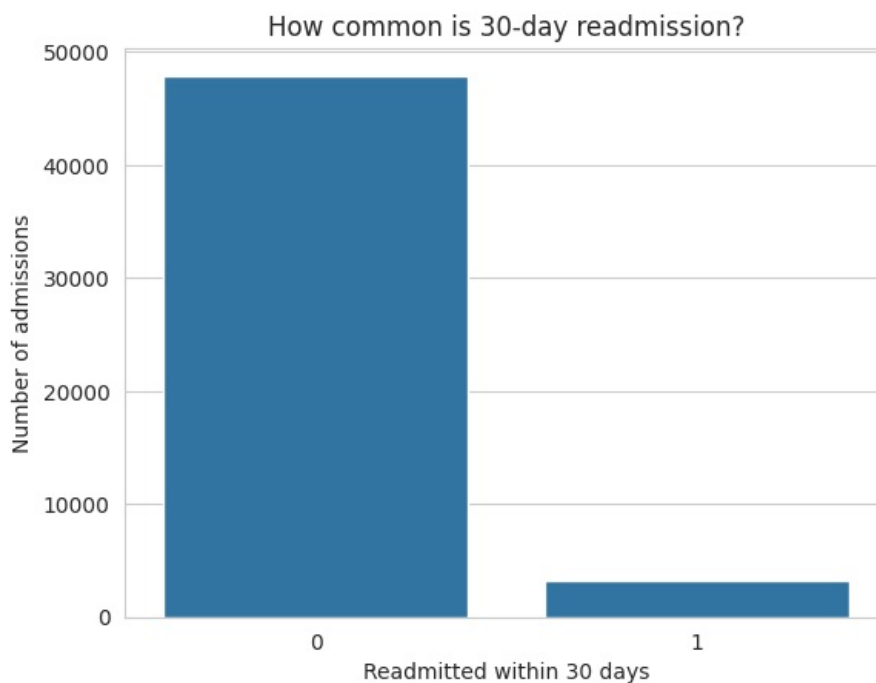
4. Look at the label before doing anything else

Before building any model, it's worth checking how common readmission actually is in this data. Most patients in a hospital dataset like this do *not* come back within 30 days, so we'd expect this to be what's called an **imbalanced** outcome - one class (no readmission) is much more common than the other (readmission). This matters a lot for how we judge the model later, so it's worth seeing the actual split now.

```
In [5]: print(df['readmit_30d'].value_counts(normalize=True).round(3))

sns.countplot(data=df, x='readmit_30d')
plt.title('How common is 30-day readmission?')
plt.xlabel('Readmitted within 30 days')
plt.ylabel('Number of admissions')
plt.show()
```

```
readmit_30d
0    0.937
1    0.063
Name: proportion, dtype: Float64
```



```
In [6]: # Clean up the data before modeling:
# - drop rows missing age or lab info (a small number of rows, safe to remove here)
# - group rare diagnoses into 'Other' so we don't end up with hundreds of tiny categories
df_clean = df.dropna(subset=['age', 'lab_abnormal_pct']).copy()

TOP_N_DIAGNOSES = 15
```

```

top_dx = df_clean['primary_diagnosis'].value_counts().nlargest(TOP_N_DIAGNOSES).index
df_clean['primary_diagnosis_grouped'] = df_clean['primary_diagnosis'].where(
    df_clean['primary_diagnosis'].isin(top_dx), other='Other'
)

print(f"Rows after cleaning: {len(df_clean):,} (dropped {len(df) - len(df_clean):,})")
df_clean['primary_diagnosis_grouped'].value_counts().head(TOP_N_DIAGNOSES + 1)

```

Rows after cleaning: 50,532 (dropped 581)

```

Out[6]: primary_diagnosis_grouped
Other                                34774
Cnrry athrscl native vssl           3449
Septicemia NOS                      2065
Subendo infarct, initial             1735
Aortic valve disorder                1134
Acute respiratory failure             1120
Intracerebral hemorrhage             1021
Unknown                             747
Pneumonia, organism NOS              718
Food/vomit pneumonitis               654
Mitral valve disorder                 562
CHF NOS                             558
Subarachnoid hemorrhage              525
Acute kidney failure NOS              520
AMI inferior wall, init               477
AMI anterior wall, init               473
Name: count, dtype: int64

```

5. Turn the data into numbers a model can use

Models can only work with numbers, but several of our features are text categories (like admission type or diagnosis). We handle this with **one-hot encoding**: each category gets turned into its own yes/no (1/0) column. For example, `admission_type` becomes separate columns like `admission_type_EMERGENCY` and `admission_type_URGENT`, each either 1 or 0.

```

In [7]: categorical_cols = ['admission_type', 'gender', 'primary_diagnosis_grouped']
numeric_cols = ['age', 'prior_admission_count', 'lab_abnormal_pct', 'total_labs']

X = pd.get_dummies(df_clean[categorical_cols + numeric_cols], columns=categorical_cols, drop_first=True)
y = df_clean['readmit_30d'].astype(int)

print(f"Feature matrix shape: {X.shape}")
X.head()

```

Feature matrix shape: (50532, 22)

```

Out[7]:
   age  prior_admission_count  lab_abnormal_pct  total_labs  admission_type_EMERGENCY  admission_type_U
0    34                      0          0.295567         203                      False
1    30                      0          0.418782         394                      True
2    30                      1          0.400631         317                      True
3    30                      2          0.338333         600                      True
4    36                      1          0.142857          91                      True

```

5 rows x 22 columns



```

In [8]: # Split the data into a training set (used to fit the model) and a test set
# (held back to check how well it works on data it hasn't seen).
# stratify=y makes sure both sets have roughly the same readmission rate,
# so the test set is a fair representation of the whole dataset.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)

```

```
# Scaling puts every numeric feature on a similar range (mean 0, standard
# deviation 1). Without this, a feature like "total labs" (values in the
# hundreds) could dominate a feature like "age" (values in the tens) just
# because of its size, not because it's actually more important.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Train: {X_train.shape}, Test: {X_test.shape}")
print(f"Train readmission rate: {y_train.mean():.3f} | Test readmission rate: {y_test.mean():.3f}")
```

Train: (40425, 22), Test: (10107, 22)
Train readmission rate: 0.063 | Test readmission rate: 0.063

6. Start with a simple baseline: Logistic Regression

Before reaching for a neural network, it's good practice to try a simple, well-understood model first. **Logistic regression** predicts a probability between 0 and 1 (here, the chance of readmission) using a weighted combination of the input features - each feature gets a weight, and the model adds them up. It's easy to inspect afterward, which makes it a good baseline to compare a more complex model against. If the neural network can't beat this, the extra complexity isn't buying us anything.

```
In [9]: # class_weight='balanced' tells the model to pay more attention to the
# minority class (readmissions). Without it, the model could get a
# deceptively high accuracy just by predicting "no readmission" for almost
# everyone, since that's the more common outcome.
logreg = LogisticRegression(max_iter=1000, class_weight='balanced', random_state=SEED)
logreg.fit(X_train_scaled, y_train)

logreg_probs = logreg.predict_proba(X_test_scaled)[: , 1]
logreg_auc = roc_auc_score(y_test, logreg_probs)
print(f"Logistic Regression AUROC: {logreg_auc:.3f}")
```

Logistic Regression AUROC: 0.657

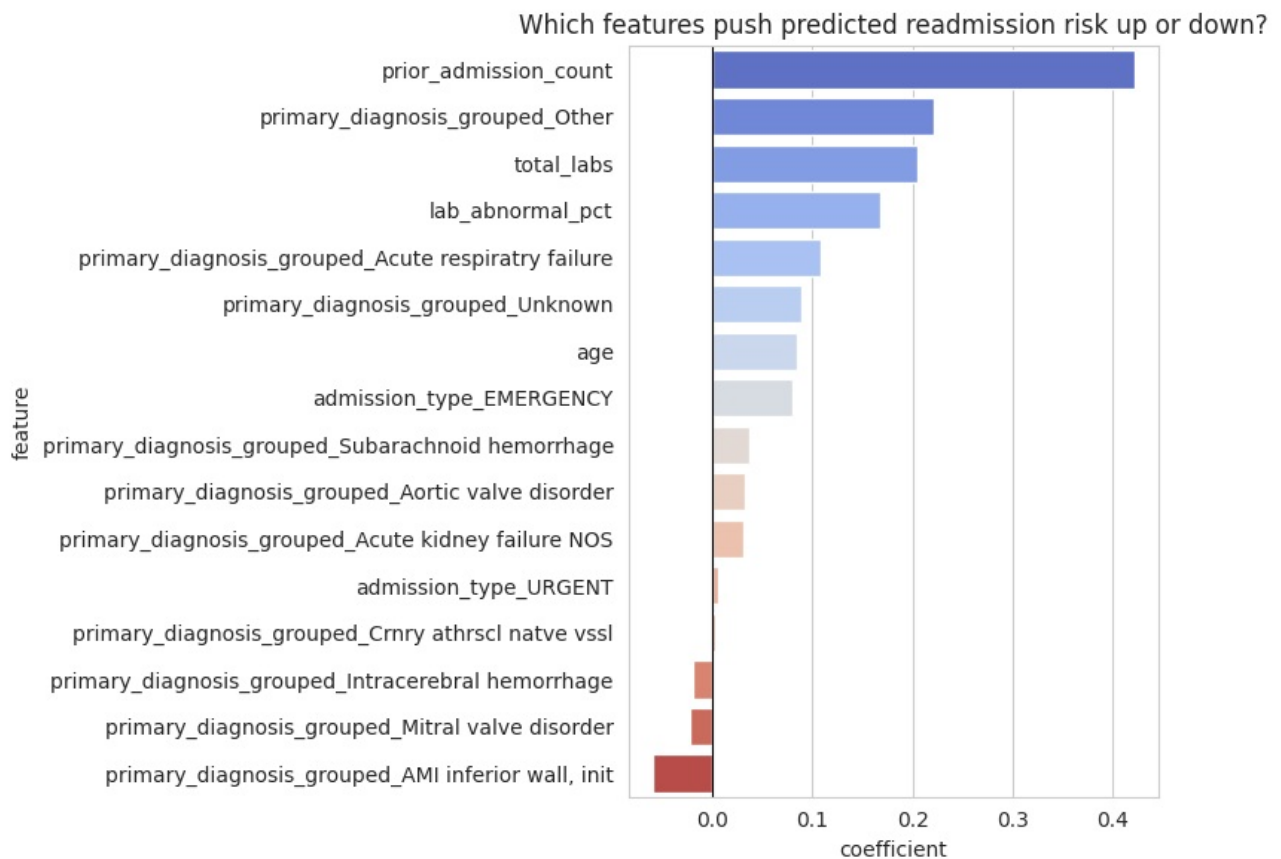
```
In [10]: # Each feature has a coefficient (weight) in a logistic regression model.
# A positive coefficient means that feature pushes the predicted risk up;
# a negative one pushes it down. Plotting the biggest ones gives us a quick
# read on what the model is actually picking up on.
coef_df = pd.DataFrame({
    'feature': X.columns,
    'coefficient': logreg.coef_[0]
}).sort_values('coefficient', ascending=False)

plt.figure(figsize=(8, 6))
top_features = pd.concat([coef_df.head(8), coef_df.tail(8)])
sns.barplot(data=top_features, y='feature', x='coefficient', palette='coolwarm')
plt.title('Which features push predicted readmission risk up or down?')
plt.axvline(0, color='black', linewidth=0.8)
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_5513/1294832729.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=top_features, y='feature', x='coefficient', palette='coolwarm')
```



7. Now try a small neural network

A neural network is a model made of layers of connected "nodes," where each layer learns to combine the outputs of the layer before it. This lets it pick up on more complicated patterns than logistic regression can - for example, combinations of features that only matter together, not on their own.

We're deliberately keeping this network small. Here's the reasoning behind each choice, since a reviewer can't see this just by looking at the numbers:

Why only two hidden layers, with 32 and then 16 nodes: after one-hot encoding, our feature table only has around 20 to 30 columns. A network with hundreds of nodes per layer would have far more adjustable parameters than we have data to support, which usually means the model starts memorizing the training examples instead of learning patterns that generalize to new patients - this is called **overfitting**. Two modest-sized layers are enough to combine our features in useful ways without falling into that trap.

Why dropout, and why 0.3 then 0.2: **dropout** is a technique where, during training, the network randomly "turns off" a fraction of its nodes on each pass. This stops the model from relying too heavily on any single node and tends to make it more robust. We use a higher dropout rate (0.3) right after the first layer, and a lower one (0.2) after the second. The idea is to apply the strongest regularization early, forcing the network to learn sturdier patterns from the raw features, and ease off slightly as we get closer to the final prediction. This is

especially useful here because readmissions are the minority class - without enough regularization, the model can end up fitting to noise in that smaller group of examples rather than a real pattern.

```
In [11]: def build_model(input_dim):
    model = models.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(32, activation='relu'),
        layers.Dropout(0.3),          # stronger dropout early on, since overfitting to
                                     # the minority (readmit) class is the main risk here
        layers.Dense(16, activation='relu'),
        layers.Dropout(0.2),          # lighter dropout closer to the final prediction
        layers.Dense(1, activation='sigmoid') # outputs a single probability between 0 and 1
    ])
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        # We track two versions of "area under the curve": AUROC (curve='ROC') and
        # AUPRC (curve='PR'). Both are explained in Section 8 - short version, AUPRC
        # is the tougher, more honest metric when one class is much rarer than the other.
        metrics=[
            tf.keras.metrics.AUC(name='auROC', curve='ROC'),
            tf.keras.metrics.AUC(name='auPRC', curve='PR')
        ]
    )
    return model

nn_model = build_model(X_train_scaled.shape[1])
nn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	736
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_2 (Dense)	(None, 1)	17

Total params: 1,281 (5.00 KB)

Trainable params: 1,281 (5.00 KB)

Non-trainable params: 0 (0.00 B)

```
In [12]: # Just like class_weight in logistic regression, this gives the model a reason
# to pay more attention to the rarer, readmitted class instead of mostly
# learning from the more common "not readmitted" examples.
n_pos = y_train.sum()
n_neg = len(y_train) - n_pos
class_weight = {0: 1.0, 1: n_neg / n_pos}

# Early stopping watches validation AUPRC during training and stops once it
# stops improving for 5 rounds (epochs), then restores the best version of
# the model seen so far. This helps prevent training for too long and
# overfitting to the training data.
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_auPRC', mode='max', patience=5, restore_best_weights=True
)

history = nn_model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    class_weight=class_weight,
    callbacks=[early_stop],
```



```

    verbose=1
)

```

```

Epoch 1/50
506/506 ————— 3s 3ms/step -auprc: 0.0868 - auroc: 0.5753 - loss: 1.3216 - val_auprc: 0.13
03 - val_auroc: 0.6523 - val_loss: 0.6980
Epoch 2/50
506/506 ————— 1s 3ms/step -auprc: 0.1092 - auroc: 0.6242 - loss: 1.2575 - val_auprc: 0.13
59 - val_auroc: 0.6590 - val_loss: 0.6829
Epoch 3/50
506/506 ————— 2s 4ms/step -auprc: 0.1181 - auroc: 0.6431 - loss: 1.2463 - val_auprc: 0.13
85 - val_auroc: 0.6612 - val_loss: 0.6951
Epoch 4/50
506/506 ————— 2s 3ms/step -auprc: 0.1240 - auroc: 0.6469 - loss: 1.2397 - val_auprc: 0.13
92 - val_auroc: 0.6651 - val_loss: 0.6727
Epoch 5/50
506/506 ————— 1s 3ms/step -auprc: 0.1264 - auroc: 0.6521 - loss: 1.2371 - val_auprc: 0.14
04 - val_auroc: 0.6648 - val_loss: 0.6703
Epoch 6/50
506/506 ————— 1s 3ms/step -auprc: 0.1235 - auroc: 0.6591 - loss: 1.2307 - val_auprc: 0.14
08 - val_auroc: 0.6672 - val_loss: 0.6746
Epoch 7/50
506/506 ————— 1s 3ms/step -auprc: 0.1285 - auroc: 0.6603 - loss: 1.2247 - val_auprc: 0.14
01 - val_auroc: 0.6653 - val_loss: 0.6782
Epoch 8/50
506/506 ————— 3s 3ms/step -auprc: 0.1273 - auroc: 0.6605 - loss: 1.2257 - val_auprc: 0.13
92 - val_auroc: 0.6658 - val_loss: 0.6780
Epoch 9/50
506/506 ————— 1s 3ms/step -auprc: 0.1315 - auroc: 0.6613 - loss: 1.2241 - val_auprc: 0.13
88 - val_auroc: 0.6663 - val_loss: 0.6776
Epoch 10/50
506/506 ————— 2s 3ms/step -auprc: 0.1333 - auroc: 0.6677 - loss: 1.2189 - val_auprc: 0.13
97 - val_auroc: 0.6686 - val_loss: 0.6724
Epoch 11/50
506/506 ————— 2s 3ms/step -auprc: 0.1298 - auroc: 0.6658 - loss: 1.2210 - val_auprc: 0.13
97 - val_auroc: 0.6679 - val_loss: 0.6725

```

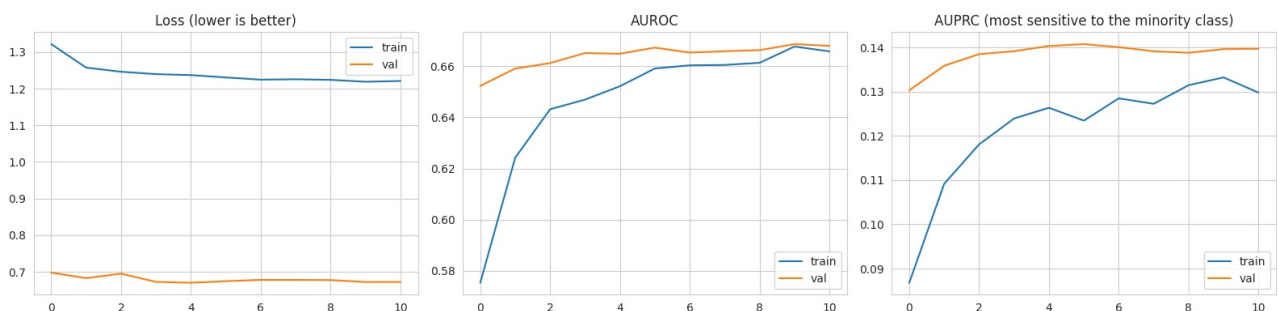
```

In [13]: # These plots let us check the model's progress during training. If the
# training line keeps improving while the validation line flattens out or
# gets worse, that's a sign of overfitting. Watch the AUPRC plot especially -
# it's the most sensitive to how well the model handles the minority class.
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
axes[0].plot(history.history['loss'], label='train')
axes[0].plot(history.history['val_loss'], label='val')
axes[0].set_title('Loss (lower is better)')
axes[0].legend()

axes[1].plot(history.history['auroc'], label='train')
axes[1].plot(history.history['val_auroc'], label='val')
axes[1].set_title('AUROC')
axes[1].legend()

axes[2].plot(history.history['auprc'], label='train')
axes[2].plot(history.history['val_auprc'], label='val')
axes[2].set_title('AUPRC (most sensitive to the minority class)')
axes[2].legend()
plt.tight_layout()
plt.show()

```



8. Compare the two models

To judge these models fairly, we look at two related but different scores, plus one metric we deliberately don't rely on:

- **Accuracy** (percent of predictions that were correct) is *not* used here as a headline number, because it's misleading on imbalanced data. In this cohort, only about 6% of admissions end in a 30-day readmission, so a model that always predicts "no readmission" would already be about 94% accurate while being completely useless.
- **AUROC** (area under the ROC curve) measures how well the model ranks patients who were readmitted above those who weren't, across all possible decision thresholds. A value of 0.5 means no better than random guessing; 1.0 means perfect ranking.
- **AUPRC** (area under the precision-recall curve) is a stricter test that focuses specifically on the readmitted (minority) group. It asks: of the patients we flag as high-risk, how many actually come back (this is called **precision**), and of all the patients who really do come back, how many did we catch (this is called **recall**)? This is closer to the question a care team would actually ask, so we treat it as the more important score of the two.

```
In [14]: nn_probs = nn_model.predict(X_test_scaled).ravel()

from sklearn.metrics import average_precision_score, precision_recall_curve

nn_auc = roc_auc_score(y_test, nn_probs)
logreg_auprc = average_precision_score(y_test, logreg_probs)
nn_auprc = average_precision_score(y_test, nn_probs)

print(f"{'Model':<22}{'AUROC':>10}{'AUPRC':>10}")
print(f"{'Logistic Regression':<22}{logreg_auc:>10.3f}{logreg_auprc:>10.3f}")
print(f"{'Neural Network':<22}{nn_auc:>10.3f}{nn_auprc:>10.3f}")
```

316/316	0s 1ms/step	
Model	AUROC	AUPRC
Logistic Regression	0.657	0.135
Neural Network	0.659	0.140

```
In [15]: fpr_lr, tpr_lr, _ = roc_curve(y_test, logreg_probs)
fpr_nn, tpr_nn, _ = roc_curve(y_test, nn_probs)

prec_lr, rec_lr, _ = precision_recall_curve(y_test, logreg_probs)
prec_nn, rec_nn, _ = precision_recall_curve(y_test, nn_probs)

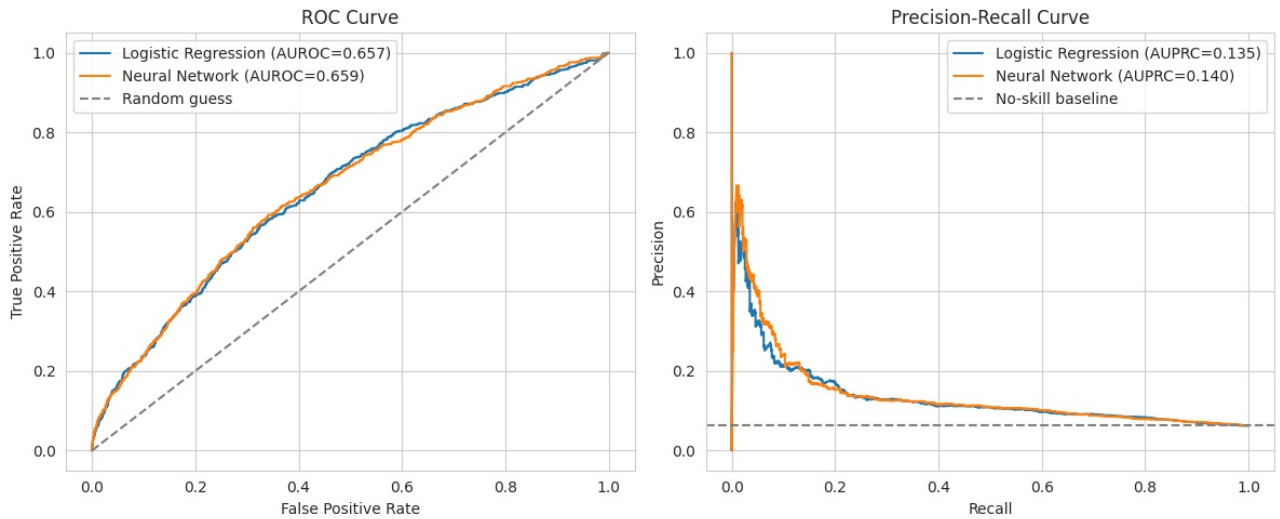
# a model with no real skill would sit at this flat line on the PR plot
baseline_rate = y_test.mean()

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(fpr_lr, tpr_lr, label=f'Logistic Regression (AUROC={logreg_auc:.3f})')
axes[0].plot(fpr_nn, tpr_nn, label=f'Neural Network (AUROC={nn_auc:.3f})')
axes[0].plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random guess')
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('ROC Curve')
axes[0].legend()

axes[1].plot(rec_lr, prec_lr, label=f'Logistic Regression (AUPRC={logreg_auprc:.3f})')
axes[1].plot(rec_nn, prec_nn, label=f'Neural Network (AUPRC={nn_auprc:.3f})')
axes[1].axhline(baseline_rate, linestyle='--', color='gray', label='No-skill baseline')
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curve')
axes[1].legend()

plt.tight_layout()
plt.show()
```



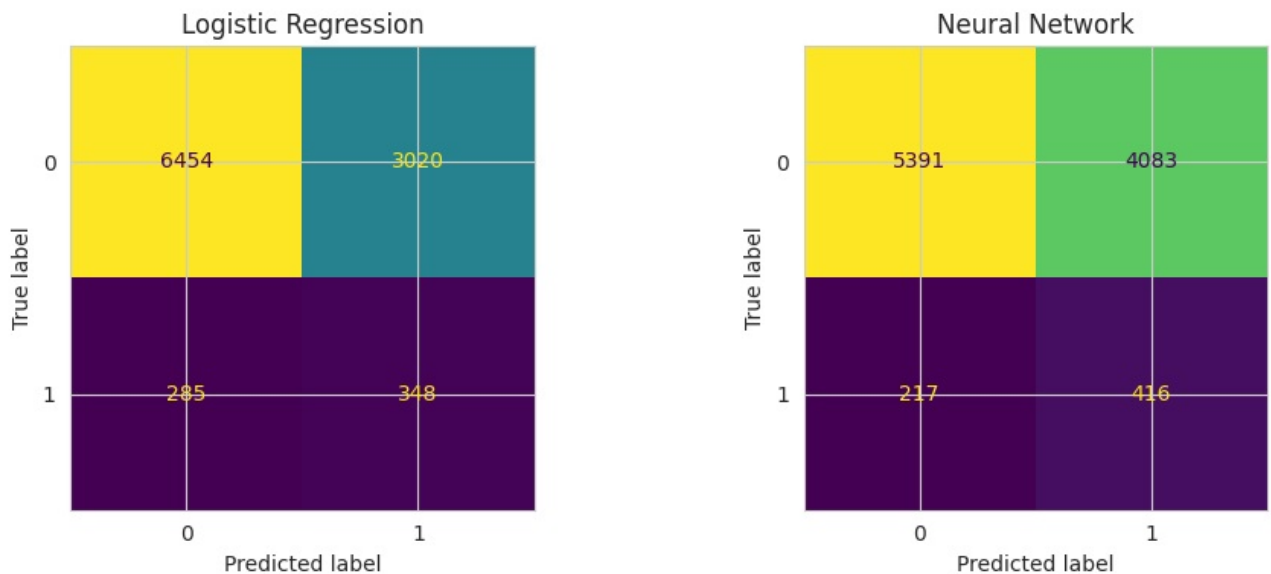
```
In [16]: # A confusion matrix shows, at a chosen cutoff (0.5 here), how many
# predictions fell into each of four buckets: correctly predicted
# readmission, correctly predicted no readmission, and the two kinds of
# mistakes (false alarms and missed readmissions). In practice, a hospital
# might move this cutoff lower than 0.5 to catch more true readmissions,
# accepting more false alarms in exchange - that's a judgment call based on
# what mistake is more costly, not something the model decides on its own.
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ConfusionMatrixDisplay(confusion_matrix(y_test, logreg_probs > 0.5)).plot(ax=axes[0], colorbar=False)
axes[0].set_title('Logistic Regression')

ConfusionMatrixDisplay(confusion_matrix(y_test, nn_probs > 0.5)).plot(ax=axes[1], colorbar=False)
axes[1].set_title('Neural Network')

plt.tight_layout()
plt.show()

print("Logistic Regression:\n", classification_report(y_test, logreg_probs > 0.5))
print("Neural Network:\n", classification_report(y_test, nn_probs > 0.5))
```



Logistic Regression:				
	precision	recall	f1-score	support
0	0.96	0.68	0.80	9474
1	0.10	0.55	0.17	633
accuracy			0.67	10107
macro avg	0.53	0.62	0.49	10107
weighted avg	0.90	0.67	0.76	10107
Neural Network:				
	precision	recall	f1-score	support
0	0.96	0.57	0.71	9474
1	0.09	0.66	0.16	633
accuracy			0.57	10107
macro avg	0.53	0.61	0.44	10107
weighted avg	0.91	0.57	0.68	10107

9. What we found

Readmission within 30 days is rare in this cohort - only about 6.3% of admissions come back that quickly. That's an important number on its own: it means any model here is hunting for a small minority, which is a harder task than it might sound like.

The two models performed almost identically. Logistic Regression scored 0.657 AUROC and 0.135 AUPRC; the neural network scored 0.659 AUROC and 0.140 AUPRC. Both the ranking quality (AUROC) and the minority-class score (AUPRC) are basically a tie, with the neural network only slightly ahead on AUPRC. For a compact, mostly tabular dataset like this one, that's a completely normal outcome to report - the extra complexity of a neural network doesn't always translate into extra predictive power, and here it mostly didn't. A simple, interpretable model gets you almost all the way there.

The two models do trade off differently at the default cutoff, though. At a 0.5 probability threshold, Logistic Regression catches 55% of the patients who actually get readmitted (recall) while staying 67% accurate overall. The neural network catches noticeably more - 66% of true readmissions - but its overall accuracy drops to 57%, because it flags far more patients as high-risk and gets more of those flags wrong. (This particular trade-off can shift a bit each time the network is retrained, since neural network training has some inherent randomness even with a fixed seed - the direction of the trade-off, more recall for less accuracy, is the consistent and important part, not the exact percentages.) Neither model is very *precise* about its high-risk flags either - only about 1 in 10 patients predicted as high-risk actually comes back within 30 days. That's a direct consequence of how rare readmission is: even a model with real skill will produce a lot of false alarms in absolute numbers when the thing it's predicting is uncommon. In practice, a hospital would tune this cutoff based on how costly a missed readmission is versus an unnecessary follow-up call, rather than defaulting to 0.5.

The strongest predictor by a clear margin was `prior_admission_count`. Patients with more admissions in their history were the single biggest signal for coming back again - a pattern in line with what you'd expect clinically, since a long admission history usually points to a chronic, recurring condition. Total lab draws, lab abnormality rate, and being an emergency (versus elective) admission were the next strongest positive signals.

A few specific diagnoses (like acute kidney failure, intracerebral hemorrhage, and certain valve disorders) came out as pushing predicted readmission risk down. This is worth being careful about rather than taking at face value: these are also conditions with meaningfully higher in-hospital mortality, and a patient who doesn't survive the admission can't be "readmitted." So a lower predicted readmission rate for these diagnoses may partly reflect mortality risk rather than a genuinely lower chance of returning to the hospital if the patient does survive. This is a good example of why a model's coefficients need a clinical read, not just a statistical one.

Limitations worth mentioning

- This model doesn't use any of the discharge note text, which likely contains useful signal we're leaving on the table.
- Some features (like diagnosis and lab results) come from the current admission itself. A model meant for real use would need to be more careful that it only uses information that would actually be available at the moment you'd want to act on the prediction.
- As noted above, the negative association between certain diagnoses and readmission risk is likely tangled up with mortality, which this notebook doesn't separately account for.
- We didn't do hyperparameter tuning or cross-validation here. Both would matter for a real deployment, but they add more complexity than is needed to explain the core idea in a tutorial like this one.

How to reproduce this

1. Get PhysioNet access to MIMIC-III and load it into Google BigQuery.
2. Run the SQL query in Section 3 to build the dataset.
3. Install scikit-learn and TensorFlow.
4. Run Sections 4 through 8 in order - no extra manual steps needed in between.